ТОО ”Колледж Хекслет”

Лабораторная работа №3

Параметры маршрутов, query-параметры и тело запроса

Выполнено студентом:

Дубинин Кирилл

Практическое задание

Раздел RO4_2. Параметры маршрутов, query-параметры и тело запроса.

Цель: Научиться получать данные HTTP-запроса через req.params, req.query и req.body, создавать динамические маршруты, фильтровать данные с помощью query-параметров и принимать JSON-данные для создания и изменения ресурсов.

Условие: RO4_2/lesson3/ЛР-3.pdf

Требования задания

• использовать express.json();

• получать динамический id через req.params;

• преобразовывать числовой id из строки в число;

• возвращать конкретный товар по идентификатору;

• возвращать 404, если товар отсутствует;

• использовать req.query для фильтрации;

• поддерживать минимум три query-параметра;

• поддерживать одновременное применение нескольких фильтров;

• принимать JSON через req.body;

• создавать новый товар;

• частично изменять товар;

• удалять товар по идентификатору;

• содержать пример совместного использования params, query и body.

Ход выполнения

Динамический id читается из req.params, фильтры — из req.query, изменения — из req.body. Фильтры категории, наличия и диапазона цены применяются совместно. POST, PATCH и DELETE изменяют массив. Счётчик nextId предотвращает повтор id после удаления. Маршрут settings демонстрирует три источника данных в одном запросе.

Проект: RO4_2/lesson3

Запуск из каталога проекта: npm install, затем npm start. Адрес сервера:

http://localhost:3000.

app.js

app.js

```
import express from 'express';
import { pathToFileURL, fileURLToPath } from 'node:url';
let products = [
    {
        "id": 1,
        "name": "Keyboard",
        "price": 15000,
        "category": "Accessories",
        "inStock": true
    },
    {
        "id": 2,
        "name": "Mouse",
        "price": 8000,
        "category": "Accessories",
        "inStock": true
    },
    {
        "id": 3,
        "name": "Monitor",
        "price": 85000,
        "category": "Monitors",
        "inStock": false
    },
    {
        "id": 4,
        "name": "Laptop",
        "price": 420000,
        "category": "Computers",
        "inStock": true
    },
    {
        "id": 5,
        "name": "Webcam",
        "price": 22000,
```

Листинг 1. app.js (часть 1)

app.js

```
        "category": "Accessories",
        "inStock": false
    }
];
let nextId = 6;

function getAllProducts(query = {}) {
    return products.filter(product =>
        (query.category === undefined || product.category === query.category) &&
        (query.inStock === undefined || product.inStock === (query.inStock === 'true'))
&&
        (query.minPrice === undefined || product.price >= Number(query.minPrice)) &&
        (query.maxPrice === undefined || product.price <= Number(query.maxPrice))
    );
}
function findProductById(id) {
    return products.find(product => product.id === id);
}
function addProduct(data) {
    const product = { ...data, id: nextId++ };
    products.push(product);
    return product;
}
function updateProductById(id, changes) {
    const product = findProductById(id);
    if (!product) return null;
    Object.assign(product, changes);
    return product;
}
function deleteProductById(id) {
    const index = products.findIndex(product => product.id === id);
    if (index === -1) return null;
    return products.splice(index, 1)[0];
}

```

Листинг 2. app.js (часть 2)

app.js

```
70
71
72  import { AppError } from './errors/AppError.js';
73
74  async function getProducts(req, res, next) {
75      try { res.json(await getAllProducts(req.query)); }
76      catch (error) { next(error); }
77  }
78  async function getProductById(req, res, next) {
79      try {
80          const product = await findProductById(Number(req.params.id));
81          if (!product) return next(new AppError('Product not found', 404));
82          res.json(product);
83      } catch (error) { next(error); }
84  }
85  async function createProduct(req, res, next) {
86      try { res.status(201).json(await addProduct(req.body)); }
87      catch (error) { next(error); }
88  }
89  async function updateProduct(req, res, next) {
90      try {
91          const product = await updateProductById(Number(req.params.id), req.body);
92          if (!product) return next(new AppError('Product not found', 404));
93          res.json(product);
94      } catch (error) { next(error); }
95  }
96  async function deleteProduct(req, res, next) {
97      try {
98          const product = await deleteProductById(Number(req.params.id));
99          if (!product) return next(new AppError('Product not found', 404));
100         res.json(product);
101     } catch (error) { next(error); }
102 }
103 function productSettings(req, res) {
104     res.json({ id: Number(req.params.id), notify: req.query.notify === 'true', changes:
```

Листинг 3. app.js (часть 3)

app.js

```
req.body });
}



import { notFound } from './middleware/notFound.js';
import { errorHandler } from './middleware/errorHandler.js';
const app = express();
app.use(express.json());
app.get('/', (req, res) => res.send('Internet Shop API'));
app.get('/about', (req, res) => res.send('Учебный REST API магазина компьютерной
техники'));
app.get('/api', (req, res) => res.json({ name: 'Internet Shop API', version: '1.0',
resources: ['/api/products', '/api/categories', '/api/orders'] }));
app.get('/api/status', (req, res) => res.json({ status: 'ok', message: 'Server is
running', requestTime: req.requestTime }));
app.get('/api/shop', (req, res) => res.json({ name: 'Kirill Tech Store', description:
'Учебный интернет-магазин', currency: 'KZT' }));
app.get('/api/products', getProducts);
app.get('/api/products/:id', getProductById);
app.post('/api/products', createProduct);
app.patch('/api/products/:id', updateProduct);
app.delete('/api/products/:id', deleteProduct);
app.patch('/api/products/:id/settings', productSettings);
app.get('/api/categories', (req, res) => res.json(['Computers', 'Accessories',
'Monitors']));
app.post('/api/categories', (req, res) => res.status(201).json({ message: 'POST route:
category creation' }));
app.get('/api/orders', (req, res) => res.json([]));
app.post('/api/orders', (req, res) => res.status(201).json({ message: 'POST route: order
 creation' }));
app.use(notFound);
app.use(errorHandler);
export default app;

```

Листинг 4. app.js (часть 4)

app.js

```
if (process.argv[1] && import.meta.url === pathToFileURL(process.argv[1]).href) {
    const PORT = Number(process.env.PORT || 3000);
    app.listen(PORT, () => console.log(`Server: http://localhost:${PORT}`));
}
```

Листинг 5. app.js (часть 5)

Проверка результата

Проверено: 05.11.2026 13:44 (Алматы). Node.js v26.12.1.

Приведены фактические результаты проверки. Полный журнал: verification.json.

• Обязательные endpoints

• HTTP-статусы и содержимое ответов

• Полный CRUD-цикл

• Совместная фильтрация

• Уникальные id после удаления

HTTP-запросы

GET / → 200

GET /about → 200

GET /api/status → 200

GET /api/shop → 200

GET /api/products → 200

GET /api/categories → 200

GET /api → 200

GET /api/orders → 200

GET /api/products/3 → 200

GET /api/products/999 → 404

GET /api/products?category=Accessories&inStock=true&minPrice=10000&maxPrice=50000 → 200

GET /api/products?inStock=false → 200

POST /api/products → 201

GET /api/products/6 → 200

PATCH /api/products/6 → 200

PATCH /api/products/4/settings?notify=true → 200

DELETE /api/products/6 → 200

GET /api/products/6 → 404

PATCH /api/products/6 → 404

DELETE /api/products/6 → 404

POST /api/products → 201

GET /api/unknown → 404

Примеры ответов

POST /api/products → 201


```
{
 "name": "SSD 2TB",
 "price": 65000,
 "category": "Components",
 "inStock": true,
 "id": 6
}
```


PATCH /api/products/6 → 200


```
{
 "name": "SSD 2TB",
 "price": 42000,
 "category": "Components",
 "inStock": false,
 "id": 6
}
```


DELETE /api/products/6 → 200


```
{
 "name": "SSD 2TB",
 "price": 42000,
 "category": "Components",
 "inStock": false,
 "id": 6
}
```


Результат

Проверены обязательные операции и указанные ошибочные сценарии; результаты сохранены вместе с исходным кодом.